Requirement 3: LLM Access to Substrate Content as Standalone Architectural Commitment in the Coordination Knowledge Substrate Pattern

Author: Wenxin Li (Independent Researcher)
ORCID: 0009-0004-8065-3235
Date: May 2, 2026

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize the third of the three minimal requirements named in the source paper's tool-agnosticism commitment — **LLM access to substrate content** — as a standalone architectural commitment with independent operational content, separable from the persistent-structured-state and human-read/write-access requirements with which it composes.

Abstract

The CKS pattern's tool-agnosticism commitment names three minimal requirements that any host environment must satisfy to instantiate the pattern: persistent structured state, human read/write access, and LLM access to substrate content. A separate note formalizes the joint commitment; companion notes formalize Requirements 1 and 2 as standalone. This note formalizes Requirement 3 as having independent operational content. The motivation is concrete: host environments routinely supply LLM access through specialized AI runtimes, AI-gateway layers, fine-tuned model deployments, and retrieval-only integrations, each of which can grant LLMs the capability to read and write substrate content while breaking the architectural commitment to standard host operations. The note states the five operational components Requirement 3 imposes, specifies what it does not require, distinguishes it from four adjacent host-capability patterns commonly conflated with it, names the load-bearing connections to downstream commitments (the AI-as-substrate-mediator role; Properties A and B of the mediator decomposition), enumerates ten failure modes, and provides an operational test for whether a given host environment satisfies the requirement separable from the broader tool-agnosticism commitment.

1. Why Requirement 3 needs to be formalized as standalone

The CKS pattern's tool-agnosticism commitment names three minimal requirements together: persistent structured state, human read/write access, and LLM access to substrate content (§7.1 of the source paper). The joint framing is correct as far as it goes, but it leaves a class of cases architecturally underspecified. Host environments routinely supply LLM access through specialized infrastructure — AI runtimes serving fine-tuned models against substrate content, AI-gateway layers mediating LLM calls, agent-framework adapters translating substrate reads into framework-native primitives, retrieval-only integrations reaching substrate exclusively through embedding-based search. Each of these can grant LLMs the capability to read and write substrate content while gating that capability behind AI-specific infrastructure that the host's other tenants do not need. Treated only as a component of a composite commitment, such environments can claim to satisfy tool-agnosticism on the strength of LLM access alone, even when the access path violates the architectural commitment the parent note carries.

The remedy is to name Requirement 3 as a standalone architectural commitment with independent operational content. The component that matters — LLM access through the same standard read/write operations the host provides for non-LLM access — is what makes the AI-as-substrate-mediator role of §4.1 operationally realizable in any tool meeting the three minimal requirements rather than only in tools deploying specific AI infrastructure. A second motivation is strategic: patentable derivations focused on LLM-substrate integration architectures — AI-gateway designs, fine-tuned model deployments, LLM-runtime systems, agent-framework adapters — are more defensibly contested when Requirement 3 is publicly formalized as standalone, because any such "innovation" can be evaluated against the specific commitment to standard operations. A third is the connection to the mediator role itself: without Requirement 3 as host-capability foundation, the role becomes nominal — LLMs may be designated as mediators in deployment configuration but cannot operationally read from or write to substrate through standard operations the host provides.

2. The Requirement 3 commitment, defined precisely

In the CKS pattern, a host environment satisfies **Requirement 3** if and only if all of the following hold during the substrate's existence on that host. The requirement has five operational components.

(a) LLMs operating within cells can read substrate content as primary input through the standard read operations the host provides. The same read mechanism humans use under Requirement 2 is the mechanism LLMs use; specialized AI-runtime read paths are not architectural preconditions. The "as primary input" qualifier carries the substrate-as-authoritative-source commitment Property A of the mediator decomposition specifies: LLMs in cells answer coordination questions by reading substrate state, not from parametric memory or session-only context.

(b) LLMs operating within cells can write substrate content through the standard write operations the host provides. The same write mechanism humans use is the mechanism LLMs use; specialized AI-runtime write paths are not architectural preconditions. LLM writes happen under the orchestration-rule authorization Property B of the mediator decomposition specifies, but the

host-capability commitment is to writes being possible through standard operations regardless of which rules govern them.

(c) Substrate content reaches LLMs in a form that preserves substrate state's authoritative meaning. Embedded representations, derived views, summaries, and other transformations may be supplied alongside substrate content for processing convenience, but the architectural commitment requires direct access to the underlying substrate state. An LLM operating only over embedded representations or LLM-curated views fails Requirement 3, because it is reading something derived from substrate state with the derivation's fidelity unverified at the architectural layer.

(d) LLM access to substrate is governable through the same architectural mechanisms that govern human access. The host's access controls, authority structure, and architectural-property qualifier on governance apply to LLM access in the same way as to human access. LLMs are subjects of access control, not exceptions to it. A host that grants LLMs special permissions or access paths not available to humans, with the LLM-specific permissions serving as preconditions of substrate access, fails this component.

(e) The host does not require specialized AI infrastructure as a precondition for LLM access. A deployment may use LLM runtimes, fine-tuned model serving, AI gateways, or agent-framework adapters operationally, but the host's architectural commitment is to standard read/write operations being sufficient. A host that requires AI-specific infrastructure for LLM access fails Requirement 3 because the property that any host meeting the three requirements works — the property the joint tool-agnosticism formalization carries — is broken at the LLM-access axis.

The five components together define what Requirement 3 requires architecturally. A host satisfying fewer than five fails the requirement, even if it supplies LLM access in some other sense.

3. What Requirement 3 does NOT require

The standalone treatment is not maximalist. Stating precisely what Requirement 3 does not require keeps the standalone framing from drifting into something stronger than the source paper supports.

It does not specify which LLM technology operates in cells. The architecture is model-agnostic; any LLM that can read content from the host and produce content that can be written to the host satisfies the requirement from the LLM side. The host's commitment is to supporting LLM access through standard operations, regardless of model choice.

It does not require LLMs to read all substrate content. LLMs operating within cells read content within the cell's bounded scope; broader access is neither required nor permitted. Requirement 3 is about host capability for LLM access within authorized scope, not universal LLM access.

It does not require a specific protocol. The host may expose substrate access through SQL, REST, file I/O, programmatic APIs, or any other standard mechanism. What the architecture requires is that the access mechanism is standard for the host — the same mechanism humans use, not a specialized AI-only path.

It does not forbid optimization layers between LLMs and substrate. Caching, query optimization, batching, and other performance infrastructure may exist between the LLM and the underlying substrate, provided the architectural access mechanism remains standard. What would violate the requirement is an optimization layer being a precondition adding AI-specific requirements beyond the three minimal requirements.

It does not forbid retrieval systems or embedding-based search. The LLM may use retrieval (RAG) or embedding search as supplementary tools alongside direct substrate access. What Requirement 3 forbids is retrieval being the only path to substrate content; the LLM must be able to read substrate state directly when orchestration rules require it.

It does not require LLM access to be high-bandwidth or low-latency. Performance characteristics are deployment concerns; the architectural commitment is to access through standard operations, not to specific performance properties.

4. What Requirement 3 is NOT

Four adjacent host-capability patterns are commonly conflated with Requirement 3. Each is a real and reasonable design choice in some other context; naming what Requirement 3 is not is what prevents the misreading.

Not LLM-specific runtimes. Some hosts provide specialized LLM runtimes — model-serving infrastructure, AI gateways, vector databases optimized for LLM access — and treat these as the LLM access path. LLM-specific runtimes satisfy Requirement 3 only if the LLM can also access substrate through the standard operations the host provides for non-LLM access. A host where the LLM-specific runtime is the only access path fails the requirement.

Not AI-gateway layers. Some deployments use AI gateways that mediate between LLMs and underlying systems, providing authentication, rate limiting, observability, and policy enforcement. AI gateways are deployment layers; they do not by themselves violate Requirement 3 if the underlying access is through standard operations. What violates the requirement is an AI gateway being the architectural precondition for LLM access — i.e., the host's access mechanism is "go through the AI gateway." Standard access through the gateway as a deployment layer is admissible; the gateway as a host requirement is not.

Not fine-tuned models with embedded substrate knowledge. Some deployments fine-tune LLMs on substrate content and treat the fine-tuned weights as the LLM's "access" to substrate. Fine-tuning is a deployment technique; it does not by itself satisfy Requirement 3, because the LLM is not reading substrate content at execution time. Fine-tuned models satisfy Requirement 3 only if they also read substrate at execution time through standard operations; using fine-tuning as the primary access mechanism fails the requirement.

Not retrieval-only LLM integration. Some deployments configure LLMs to access substrate exclusively through retrieval systems — vector search, embedding-based queries, or RAG pipelines. Retrieval-only access satisfies Requirement 3 only if retrieval supplements direct access rather than

replacing it. The LLM must be able to read substrate state directly when orchestration rules require it; retrieval as the only path fails the requirement.

The four distinctions together keep Requirement 3's content precise: the requirement is host capability for LLM access through standard operations, not the commitment that LLM access exists in some other sense.

5. Why Requirement 3 is load-bearing for downstream commitments

Requirement 3 is the host-capability foundation on which several CKS commitments rest. The AI-as-substrate-mediator role of §4.1 is operationally realizable only because Requirement 3 grants this foundation; without it the role becomes nominal, since LLMs may be designated as mediators in deployment configuration but cannot operationally read from or write to substrate through standard operations. Property A of the mediator decomposition — LLM reads from substrate as primary source of state — grounds in component (a) of Requirement 3. Property B — LLM writes under orchestration rules — grounds in component (b). The architectural-property qualifier on governance, which requires governance to be a property of architecture rather than of process, depends on Requirement 3 to make LLM access architectural rather than procedural: LLMs reach substrate through standard operations the host provides, not through procedural AI-infrastructure layers a deployment may or may not maintain. The joint tool-agnosticism commitment depends on Requirement 3 directly; a host satisfying Requirements 1 and 2 but lacking Requirement 3 does not support the mediator role and therefore does not support full CKS substrates. The non-specialist governance commitment depends on Requirement 3 for the property that commodity tools meeting all three minimal requirements support LLM mediation through their standard programmatic access — a property that fails the moment specialized AI infrastructure becomes a precondition. None of these are new commitments; they are the connections that follow from treating Requirement 3 as having independent operational content.

6. Failure modes that violate Requirement 3

A host environment can fail Requirement 3 specifically, even when it satisfies Requirements 1 and 2 and provides LLM access in some other sense. Ten failure modes name the most common ways this happens in 2024–2026 deployments.**(a) LLM-only runtime.** The host provides LLM access only through specialized model-serving infrastructure or AI runtimes; humans access substrate through different mechanisms. The LLM cannot use the host's standard read/write operations.

(b) AI-gateway as architectural precondition. The host requires LLM access to flow through an AI gateway as the only path; standard operations are not available to LLMs. The gateway has been promoted from deployment layer to architectural requirement.

(c) Fine-tuning as primary access. The deployment fine-tunes LLMs on substrate content and configures the LLM to operate from fine-tuned knowledge rather than execution-time substrate reads. The LLM is operating from parametric memory of past substrate content, not reading current substrate state.

(d) Retrieval-only access. The deployment configures LLM access exclusively through retrieval systems; direct substrate state is not reachable from the LLM's execution context. The LLM cannot read substrate state when orchestration rules require it.

(e) Embedded-only LLM access. The host provides LLM access only to embedded representations of substrate content (vector representations, semantic indexes); direct substrate state in inspectable form is unreachable from the LLM's execution context.

(f) Specialized-permission access. The host's permission system grants LLMs special permissions or access paths not available to humans, with the LLM-specific permissions being preconditions of substrate access. The LLM's access is qualitatively different from human access.

(g) AI-specific schema requirements. The host requires substrates to be structured in AI-specific schemas — vector-embedded fields, ML-feature-formatted records, AI-optimized layouts — to be accessible by LLMs. The LLM-access component imposes schema requirements beyond what Requirement 1 carries.

(h) Vendor-LLM coupling. The host's LLM access works only with specific LLM vendors or model families; switching models requires changing the host's access infrastructure. The architectural commitment to model-agnosticism is broken.

(i) Asymmetric read/write access. The host grants LLMs read access through standard operations but routes LLM writes through specialized validation, gating, or transformation layers not available for human writes. Read direct, write mediated — half of the requirement satisfied, half failing.

(j) Cell-runtime-locked LLM access. The host's LLM access works only through specific cell runtime infrastructure — a particular agent framework, a specific orchestration engine, a vendor-supplied execution environment. Other cell implementations cannot use standard operations to enable LLM mediation.

A host exhibiting any of (a)–(j) does not satisfy Requirement 3 specifically, and naming the failure precisely is what allows downstream remediation.

7. Operational test

A host environment satisfies Requirement 3 if and only if all of the following are true at all times during the substrate's existence on the host:

1. LLMs operating within cells can read substrate content through the standard read operations the host provides; specialized AI runtimes are not architectural preconditions.
2. LLMs operating within cells can write substrate content through the standard write operations the host provides; specialized AI runtimes are not architectural preconditions for writes.
3. Substrate content reaches LLMs in a form that preserves substrate state's authoritative meaning; direct access to underlying state is available, not only embedded representations or LLM-curated views.

4. LLM access is governed through the same architectural mechanisms that govern human access; the host's access controls, authority structure, and architectural-property qualifier apply to LLMs in the same way as to humans.
5. The host does not require specialized AI infrastructure beyond what Requirements 1 and 2 already imply; the property that any host meeting the three requirements works holds for LLM access.

A host that fails any of (1)–(5) does not satisfy Requirement 3 specifically, even if it provides LLM access in some other sense. Such a host is not CKS-coherent on the LLM-access axis, and downstream work that relies on its LLM-access guarantees should be scoped accordingly.

8. Conclusion

Implementations under pressure to optimize LLM performance, integrate with AI-specific platforms, or take advantage of vendor LLM features consistently drift toward host environments that route LLM access through specialized infrastructure. The drift is steady because each AI-specific feature offers genuine benefits — better performance, easier integration, richer observability — and the substrate appears to "still work" with the AI infrastructure in place. The architectural commitment to standard operations is what each specialized layer erodes when added as a precondition. Implementations that drift away from Requirement 3 produce systems where the substrate exists but LLM access depends on vendor-specific AI infrastructure, where switching providers requires re-architecting the host, and where commodity tools meeting Requirements 1 and 2 cannot be used because they lack the AI-specific layers the deployment requires. The downstream consequences manifest as vendor lock-in, tool-agnosticism failures, and architectural-property failures — LLM access becomes resilient only when the AI infrastructure functions correctly, which is procedural rather than architectural.

Naming Requirement 3 as standalone architectural commitment — with the five operational components, the six limitations, the four adjacent-pattern distinctions, the load-bearing connections to the mediator role and Properties A and B, and the ten failure modes specified above — gives downstream implementers a precise specification of what host environments must provide for LLM mediation to be architecturally exercisable through standard operations. Companion notes formalize Requirements 1 and 2 as standalone and the joint sufficiency and individual necessity of the three-requirement set, completing the tool-agnosticism decomposition. Subsequent work that uses "Requirement 3" in a different sense is using a different concept, and the difference should be named.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems.* April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Requirement 3: LLM Access to Substrate Content as Standalone Architectural Commitment in the Coordination Knowledge Substrate Pattern.* May 2, 2026. ORCID: 0009-0004-8065-3235.